UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+
Accredited University

WORKSHEET 1 (A)

Student Name: Sahil Sharma **UID: 26MCI10257**

Branch: MCA (AI & ML) **Section/Group:** 26MAM-4A

Semester: 1 **Date of Performance:** 09-08-26

Subject Name: Python Programming

Subject Code: 26CAH-606

AIM

- Write a program to print twin primes less than 1000. If two consecutive odd numbers are both prime then they are known as twin primes.

Objective

- Understanding Prime Numbers: Gain a deeper understanding of prime numbers and specifically, twin primes.
- Algorithm Development: Learn to develop algorithms that can efficiently check for prime numbers and identify twin primes within a range.

Software Requirements

- Python

Theory

A)Twin Prime Numbers: Twin primes are a pair of prime numbers whose difference is exactly 2. For example, (3, 5) and (11, 13) are twin prime pairs. A number is called prime if it has exactly two factors: 1 and itself. To find prime numbers efficiently, the Sieve of Eratosthenes can be used. It repeatedly marks the multiples of each prime number as composite. For twin primes, we check consecutive numbers i and i + 2. If both numbers are prime, they form a twin prime pair.

Source Code (A)

```
for j in range(2, 1001):
    count1 = 0

    for i in range(1, j + 1):
        if j % i == 0:
            count1 += 1

    if count1 == 2:
        count2 = 0

        for i in range(1, j + 3):
            if (j + 2) % i == 0:
                count2 += 1

        if count2 == 2 and j + 2 <= 1000:
            print(f"({j}, {j + 2})")
```

Output

```
singh@RAJAs-MacBook-Air Python % /usr/local/bin/python3 "/Users/singh/Do
cuments/BCA/Coding Languages/Python/add.py"
(3, 5)
(5, 7)
(11, 13)
(17, 19)
(29, 31)
(41, 43)
(59, 61)
(71, 73)
(101, 103)
(107, 109)
(137, 139)
(149, 151)
(179, 181)
(191, 193)
(197, 199)
(227, 229)
(239, 241)
(269, 271)
(281, 283)
(311, 313)
(347, 349)
(419, 421)
(431, 433)
(461, 463)
(521, 523)
(569, 571)
(599, 601)
(617, 619)
(641, 643)
(659, 661)
(809, 811)
(821, 823)
(827, 829)
(857, 859)
(881, 883)
```

Learning Outcome

- Prime Identification: Successfully identified and printed all twin primes less than 1000.
- Algorithm Efficiency: Implemented efficient algorithms for prime checking.
- Coding Practice: Enhanced programming skills in Python with practical algorithm implementation.

UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+
Accredited University

WORKSHEET 1 (B)

AIM

- Write a program to implement these formulae of permutations and combinations. Number of permutations of n objects taken r at a time: p(n, r) = n! / (n-r)!. Number of combinations of n objects taken r at a time is: c(n, r) = n! / (r!*(n-r)!) = p(n,r) / r!.

Objective

- Mathematical Computation: Learn how to implement mathematical formulae programmatically, specifically for permutations and combinations.
- Factorial Calculations: Understand the use of factorials in calculating permutations and combinations.
- Efficiency Considerations: Explore efficient ways to compute permutations and combinations using built-in functions like math.factorial to handle large numbers.

Theory

A) Permutations and Combinations:

A permutation is an arrangement where the order of objects matters.

$$P(n,r)=(n-r)!n!$$

A combination is a selection where the order of objects does not matter.

$$C(n,r)=n!$$

Here, n! represents the factorial of n.

Source Code

```python
def factorial(num):
    fact = 1

    for i in range(1, num + 1):
        fact = fact * i

    return fact


n = int(input("Enter Value of N: "))
r = int(input("Enter Value of R: "))

permutation = factorial(n) // factorial(n - r)
combination = factorial(n) // (factorial(r) * factorial(n - r))

print("Permutation:", permutation)
print("Combination:", combination)
```

Output

```
singh@RAJAs-MacBook-Air Python % /usr/local/bin/python3 "/Users/singh/Documents/BCA/Coding Languages/Python/add.py"
Enter Value of N: 8
Enter Value of R: 3
Permutation: 336
Combination: 56
```

Learning Outcome

- Mathematical Computations: Accurately calculated permutations and combinations using factorials.
- Algorithm Efficiency: Implemented efficient algorithms for factorial calculations.
- Concept Application: Applied mathematical concepts in programming to solve real-world problems.
- Coding Practice: Enhanced programming skills in Python with practical algorithm implementation.